

Javascript

```

    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_LOCAL_ID', OBS_LOCAL_ID);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_UID', OBS_UID);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_RECORD_STATUS_CODE', OBS_RECORD_STATUS_CODE);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_DOMAIN_CODE_ST', OBS_DOMAIN_CODE_ST);
    dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Accession_nbr', Accession_nbr);
}
}
//If no results are returned from the NBS_MSGOUTE query, create a message to only update from NBS_ODSE query results.
else {
    dbNBS_MSGOUTE_Results.text = '<message></message>';
    for (var i=1; i <= dbNBS_ODSE_Results.getRepeatCount('/message/EDX_ACTIVITY_LOG_Record'); i++)
    {
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/NBS_Interface_UID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Interface_UID));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/ELR_MESSAGE_ID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/ELR_MESSAGE_ID));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Message_Control_ID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Message_Control_ID));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/EDX_ACTIVITY_LOG_STATUS'));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_LOCAL_ID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_LOCAL_ID));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_UID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_UID));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_RECORD_STATUS_CODE', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_RECORD_STATUS_CODE'));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_DOMAIN_CODE_ST', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_DOMAIN_CODE_ST));
        dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Accession_nbr', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Accession_nbr'));
    }
}

//
=====
//_____ COMBINE QUERY RESPONSES FROM NBS_ODSE AND NBS_MSGOUTE (end
//_____
//
=====

```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR Polling NBS First Pass/16dd71dd-db28-4d38-ba60-4232b789014b,ECLIR - ELR Polling NBS First Pass/c696374e-322b-4a60-a4ed-fd64d911b687]
Match Property	InputTime
Message Set Timeout	2
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
 db ECLIR Inserts..... Database LookUp	312
Error Connector	Page
 no-op to Sink..... No-operation	320



db ECLIR Query

Database Message Extraction

On ECLIR - ELR Polling NBS First Pass route

Filter

Notes
Purpose: This Database Message Extraction filter: Returns an XML list of records from the ELR_MESSAGE table that do not yet have key NBS identifiers recorded in the ELR_MESSAGE_NBS table. A limit of TOP (250) is placed on the query to prevent excessive CPU usage on the database server. Any records that were not returned will be picked up when the route is triggered again 15 minutes later. They query only looks for the top 250 records and excludes any records that have been tried more than 24 times according to the field NBS_POLLING_ATTEMPTS in the ELR_MESSAGE_NBS table and has SUCCESS_FL = 0.

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File:

```
<statement name="ELR_MESSAGE_NBS_Record" ide:type="executeSql" ide:summary="Query to ELR_MESSAGE_NBS table for all records in the last 30 days that do not have an NBS INTERFACE_UID." ide:allowCompoundStatements="true">
  <sql>SELECT TOP(500) MAX(ELR_MESSAGE.ID) AS ELR_MESSAGE_ID,
  ELR_MESSAGE.MSH_10 AS MESSAGE_CONTROL_ID
  FROM ELR_MESSAGE
  LEFT OUTER JOIN ELR_MESSAGE_NBS ON ELR_MESSAGE.ID = ELR_MESSAGE_NBS.ELR_MESSAGE_ID
  WHERE ELR_MESSAGE.MSH_10 IS NOT NULL AND
  ELR_MESSAGE_NBS.EDX_ACTIVITY_LOG_STATUS IS NULL AND
  ELR_MESSAGE.NBS_POLLING_ATTEMPTS <= 25 AND
  ELR_MESSAGE.INSERT_DATE >= $DateRange
  GROUP BY ELR_MESSAGE.MSH_10</sql>
  <statement name="Increment Polling Attempts" ide:type="executeSql" ide:summary="Increment the number of attempts at polling for each record returned in the previous query." ide:allowCompoundStatements="true">
    <sql>UPDATE ELR_MESSAGE
    SET NBS_POLLING_ATTEMPTS = NBS_POLLING_ATTEMPTS + 1
    WHERE ID = @ELR_MESSAGE_ID</sql>
  </statement>
</statement>
</config>
```

Destinations

Name	Page
 js Create strMsgControlIDs Property for NBS_ODSE query..... JavaScript (v.1)	305
Error Connector	Page
 no-op to Sink..... No-operation	320

 js Create strInterface_UIDs Property for NBS_MSGOUTE query

Filter

JavaScript (v.1)
On ECLIR - ELR Polling NBS First Pass route

Notes

Purpose: This JavaScript filter:
Sets the message property strInterface_UIDs to be used in the Database Lookup filter *db NBS_MSGOUTE Query* to query based on the Interface_UIDs returned from the *db NBS_ODSE Query Database Message Extraction* filter.
A message collector is used to capture the results from:
js Create strMsgControlIDs Property for NBS_ODSE query JavaScript filter
db NBS_ODSE Query Database Message Extraction filter
The ELR_MESSAGE_ID element is persisted from the *js Create strMsgControlIDs Property for NBS_ODSE query* filter output and added to the *db NBS_ODSE Query* filter results.
If no results were returned from the Database Message Extraction filter *db NBS_ODSE Query*, the message is sent to the Sink *ECLIR - ELR Sink*.

Notes

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var dbNBS_ODSE_Results = output.append(input[0]);
var dbECLIR_Results = input[1];
var strInterface_UIDs = '';

dbNBS_ODSE_Results.setProperty('NoResults', false);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTY strInterface_UIDs AND PERSIST RESULTS FROM PREVIOUS QUERY (start
)
//
=====

if (dbNBS_ODSE_Results.text.indexOf('EDX_ACTIVITY_LOG_Record') != -1) {
  for (var i=1; i <= dbNBS_ODSE_Results.getRepeatCount('/message/EDX_ACTIVITY_LOG_Record'); i++)
  {
    if (strInterface_UIDs != '') {
      strInterface_UIDs += '|';
    }
    strInterface_UIDs += dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Interface_UID');
    var msgControlID = dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Message_Control_ID');
    dbNBS_ODSE_Results.setField('/message/EDX_ACTIVITY_LOG_Record['+i+']/ELR_MESSAGE_ID', dbECLIR_Results.getField("/message/ELR_MESSAGE_NBS_Record[MESSAGE_CONTROL_ID='"+ msgControlID + "']/ELR_MESSAGE_ID"));
  }

  dbNBS_ODSE_Results.setProperty('strInterface_UIDs', strInterface_UIDs);
}
//If no results are returned from the NBS_ODSE query, send to Sink.
else {
  dbNBS_ODSE_Results.setProperty('NoResults', true);
}
```

Javascript

```
//
=====
//_____ SET MESSAGE PROPERTY strInterface_UIDs AND PERSIST RESULTS FROM PREVIOUS QUERY (end
)_____
//
=====
```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR Polling NBS First Pass/3bc8d502-65da-4701-9fd3-0390a52111c4,ECLIR - ELR Polling NBS First Pass/17d4161d-1cde-4d92-b972-d74fcec960c1]
Match Property	InputTime
Message Set Timeout	2
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
Output Condition: Results Returned NoResults = 'false'	
 db NBS_MSGOUTE Query.....Database Message Extraction	307
Output Condition: Results Returned NoResults = 'false'	
 js Combine Query Responses.....JavaScript (v.1)	313
Output Condition: No Results Returned NoResults = 'true'	
 no-op to Sink.....No-operation	320

**no-op REH to Sink****No-operation**

On ECLIR - ELR Polling NBS First Pass route

Filter

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Polling NBS First Pass Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink*.

Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink..... Sink	13



no-op to Sink

Filter

No-operation
On ECLIR - ELR Polling NBS First Pass route

Notes
Purpose: This No-operation filter sends all messages to the <i>ECLIR - ELR Sink</i> after queries are completed or if empty result sets are received.

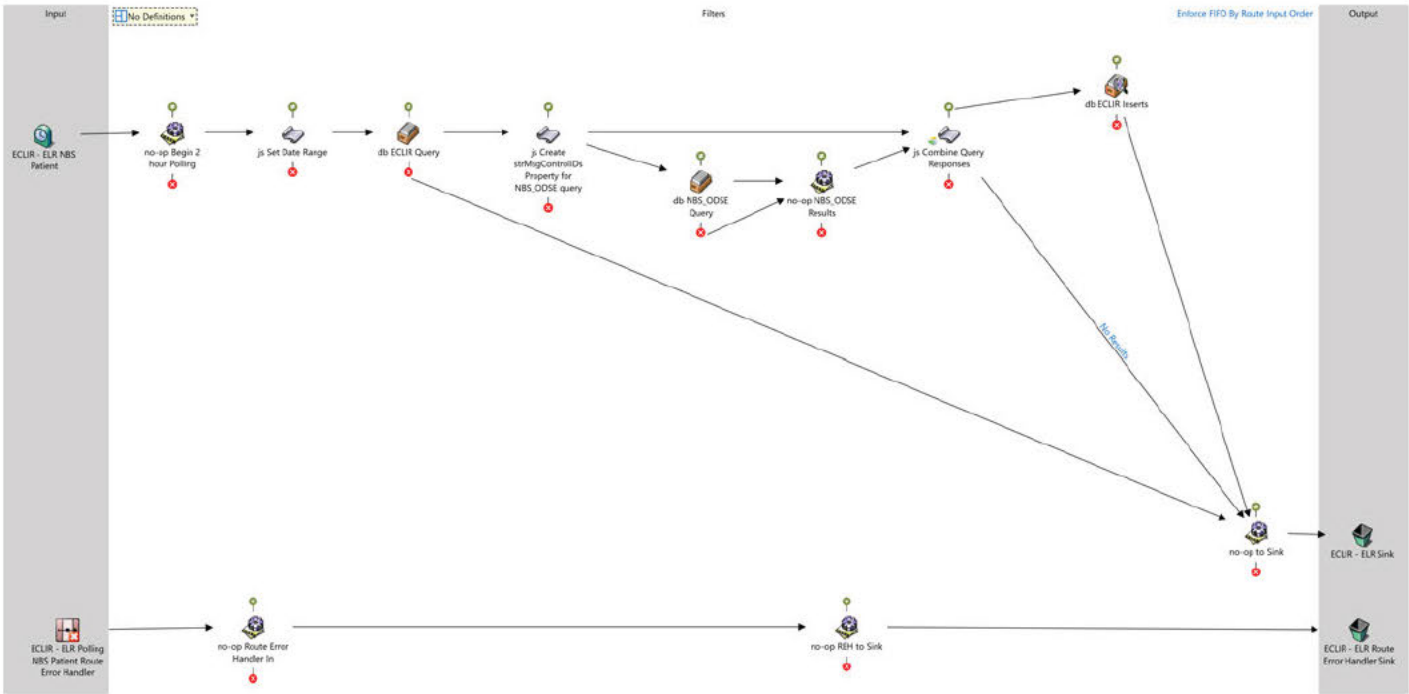
Destinations

Name	Page
 ECLIR - ELR Sink..... Sink	14



ECLIR - ELR Polling NBS Patient

Route



Notes

Purpose: This route triggers every 2 hours starting at 42 minutes past the hour to query NBS tables for key identifiers and stores them to the ECLIR database to allow messages to be linked together throughout the life of the message.

Inputs		Page
ECLIR - ELR NBS Patient.....	Timer	20
Outputs		Page
ECLIR - ELR Sink.....	Sink	14
ECLIR - ELR Route Error Handler Sink.....	Sink	13

Filters

Name	Page
db ECLIR Inserts.....	Database LookUp 328
db ECLIR Query.....	Database Message Extraction 329
db NBS_ODSE Query.....	Database Message Extraction 323
js Combine Query Responses.....	JavaScript (v.1) 326
js Create strMsgControlIDs Property for NBS_ODSE query.....	JavaScript (v.1) 332
js Set Date Range.....	JavaScript (v.1) 324
no-op Begin 2 hour Polling.....	No-operation 326

Name	Page
 no-op NBS_ODSE Results.....No-operation	331
 no-op REH to Sink..... No-operation	331
 no-op Route Error Handler In..... No-operation	326
 no-op to Sink..... No-operation	331

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



db NBS_ODSE Query

Database Message Extraction

On ECLIR - ELR Polling NBS Patient route

Filter

Notes

Purpose: This Database Message Extraction filter returns an XML list of records with key patient identifiers from the NBS_ODSE.edx_activity_log table based on the list returned from the previous filter *js Create strMsgControlIDs Property for NBS_ODSE query*.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:	[REDACTED]	

```

<statement name="EDX_ACTIVITY_LOG_Patient_Record" ide:type="executeSql" ide:summary="Query
EDX_ACTIVITY_LOG table in NBS_ODSE to retrieve key identifiers relevant to the patient." ide
:allowCompoundStatements="true">
  <sql>SELECT *
FROM
  (SELECT ROW_NUMBER() OVER (PARTITION BY e.Message_id, e.business_obj_localid ORDER BY e.edx
_activity_log_uid DESC) AS rn,
   e.Message_id, e.business_obj_localid AS obs_local_id,
   p.local_id AS person_local_id, phc.local_id AS CASE_ID
FROM [nbs_odse].[dbo].EDX_activity_log e
  INNER JOIN [nbs_odse].[dbo].Participation par ON e.target_uid = par.act_uid
  INNER JOIN [nbs_odse].[dbo].Person p ON p.person_uid = par.subject_entity_uid
  LEFT JOIN [nbs_odse].[dbo].Act_relationship a ON e.target_uid = a.source_act_uid

```

Configuration File:

```
LEFT JOIN [nbs_odse].[dbo].Public_health_case phc ON a.target_act_uid = phc.public_health
_case_uid
WHERE e.target_type_cd = 'LAB'
AND par.type_cd = 'PATSBJ'
AND e.record_status_time >= DATEADD(dd, -CAST($pollingDR AS INT), GETDATE())
AND e.Message_id IN (SELECT * FROM STRING_SPLIT($strMsgControlIDs, '|'))
) ds
WHERE ds.rn = 1</sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op NBS_ODSE Results.....No-operation	331
Error Connector	Page
 no-op NBS_ODSE Results.....No-operation	331

 js Set Date Range

Filter

JavaScript (v.1)
On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This JavaScript filter sets the message property DateRange to be used in the Database Lookup filters *db ECLIR Query* to allow the query to only select records from the last 30 days for querying.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var pollingDR = getVariable('DR_ELR_Patient');
next.text = 'ELR NBS Patient Polling';

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====
```

Javascript

```
//
=====
// _____ FUNCTIONS (start
)
//
=====

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
    num = num.toString();
    while (num.length < size) num = "0" + num;
    return num;
}

//
=====
// _____ FUNCTIONS (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTY DateRange (start
)
//
=====

var dateRange = new Date();
dateRange.setDate(dateRange.getDate() - pollingDR);
var fieldYear = dateRange.getFullYear();
var fieldMonth = fnPadZeros(dateRange.getMonth() + 1, 2);
var fieldDay = fnPadZeros(dateRange.getDate(), 2);
var fieldHours = fnPadZeros(dateRange.getHours(), 2);
var fieldMinutes = fnPadZeros(dateRange.getMinutes(), 2);
var fieldSeconds = fnPadZeros(dateRange.getSeconds(), 2);

var formatteddateRange = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+fieldMinutes
+':'+fieldSeconds;

next.setProperty('DateRange',formatteddateRange);
next.setProperty('pollingDR',pollingDR);

//
=====
// _____ SET MESSAGE PROPERTY DateRange (end
)
//
=====
```

Destinations

Name	Page
 db ECLIR Query..... Database Message Extraction	329



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Polling NBS Patient Route Error Handler* when an unhandled error occurs anywhere in the route. These errors cannot be recorded in the ELR_ERROR table because there will be multiple messages and no way to know which message caused the error.

Destinations

Name	Page
 no-op REH to Sink..... No-operation	331



no-op Begin 2 hour Polling

Filter

No-operation

On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This No-operation filter receives a trigger message every 2 hours from the *ECLIR - ELR NBS Patient Timer* to begin the polling process for ELR.

Destinations

Name	Page
 js Set Date Range..... JavaScript (v.1)	324



js Combine Query Responses

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This JavaScript filter:

Combines the results from the Database Message Extraction filter *db NBS_ODSE Query* and the output from the JavaScript filter *js Create strMsgControlIDs Property for NBS_ODSE query* into an XML file that will be used in the Database Lookup filter *db ECLIR Inserts* to insert any result data received.

A message collector is used to capture the results from:

js Create strMsgControlIDs Property for NBS_ODSE query JavaScript filter

db NBS_ODSE Query Database Message Extraction filter

The ELR_MESSAGE_ID element is persisted from the *js Create strMsgControlIDs Property for NBS_ODSE query* filter output and added to the *db NBS_ODSE Query* filter results.

If no results were returned from the *db NBS_ODSE Query* filter, the message is sent to the Sink *ECLIR - ELR Sink*.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var dbNBS_ODSE_Results = output.append(input[0]);
var dbECLIR_PATIENT_Results = input[1];
dbNBS_ODSE_Results.setProperty('NoResults', false);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ COMBINE QUERY RESPONSES FROM ECLIR AND NBS_ODSE (start
)
//
=====

if (dbNBS_ODSE_Results.text.indexOf('EDX_ACTIVITY_LOG_Patient_Record') != -1) {
  for (var i=1; i <= dbNBS_ODSE_Results.getRepeatCount('/message/EDX_ACTIVITY_LOG_Patient_Record'); i++) {
    var Message_id = dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Patient_Record[' + i + ']/Message_id');
    var ELR_MESSAGE_ID = dbECLIR_PATIENT_Results.getField("/message/ELR_MESSAGE_NBS_PATIENT_Record[MESSAGE_CONTROL_ID='"+ Message_id + "'][1]/ELR_MESSAGE_ID");
```

Javascript

```

    dbNBS_ODSE_Results.setField('/message/EDX_ACTIVITY_LOG_Patient_Record['+i+']/ELR_MESSAGE_ID',
    ELR_MESSAGE_ID);
  }
}
//If no results are returned from the NBS_ODSE query, create a message to only update from NBS
_ODSE query results.
else {
  dbNBS_ODSE_Results.setProperty('NoResults', true);
}

//
=====
//_____ COMBINE QUERY RESPONSES FROM ECLIR AND NBS_ODSE (end
)_____
//
=====

```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR Polling NBS Patient/e846be48-7214-4d21-af1a-6bfa0dd4d959 ,ECLIR - ELR Polling NBS Patient/f8b466a4-0533-4f75-bf92-287513f54071]
Match Property	InputTime
Message Set Timeout	2
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
Output Condition: No Results	
NoResults = 'true' ignoring case	
no-op to Sink..... No-operation	331
Error Connector	Page
db ECLIR Inserts..... Database LookUp	328

**db ECLIR Inserts****Database LookUp**

On ECLIR - ELR Polling NBS Patient route

Filter

Notes

Purpose: This Database Lookup filter inserts records in the ELR_MESSAGE_NBS_PATIENT table with key NBS patient identifiers.

Configuration Properties

Maximum Concurrency	1
---------------------	---

Configuration Properties		
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		
<pre><statement name="ELR_MESSAGE_NBS_PATIENT Insert" ide:type="executeSql" iterate="EDX_ACTIVITY _LOG_Patient_Record" ide:summary="Insert result data into ELR_MESSAGE_NBS_PATIENT table for the ELR_MESSAGE_ID." ide:allowCompoundStatements="true"> <sql>INSERT INTO ELR_MESSAGE_NBS_PATIENT (ELR_MESSAGE_ID ,OBS_LOCAL_ID ,PERSON_LOCAL_ID ,CASE_ID ,INSERT_DATE) VALUES (~/message/EDX_ACTIVITY_LOG_Patient_Record/ELR_MESSAGE_ID`   ,~/message/EDX_ACTIVITY_LOG_Patient_Record/obs_local_id` ,~/message/EDX_ACTIVITY_LOG_Patient_Record/person_local_id`   ,~/message/EDX_ACTIVITY_LOG_Patient_Record/CASE_ID` ,SYSDATETIME())</sql> </statement> </config></pre>		

Destinations

Name	Page
no-op to Sink..... No-operation	331

On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This Database Message Extraction filter returns an XML list of records from the ELR_MESSAGE table that do not yet have key NBS patient identifiers in the ELR_MESSAGE_NBS_PATIENT table.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Body Column		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File:

```

<statement name="ELR_MESSAGE_NBS_PATIENT_Record" ide:type="executeSql" ide:summary="Query to
ELR_MESSAGE table to retrieve all records that do not yet have a record in the ELR_MESSAGE_NBS
_PATIENT table." ide:allowCompoundStatements="true">
  <sql>SELECT ELR_MESSAGE.ID AS ELR_MESSAGE_ID,
    ELR_MESSAGE.MSH_10 AS MESSAGE_CONTROL_ID
  FROM ELR_MESSAGE
  INNER JOIN ELR_MESSAGE_NBS ON ELR_MESSAGE.ID = ELR_MESSAGE_NBS.ELR_MESSAGE_ID
  LEFT OUTER JOIN ELR_MESSAGE_NBS_PATIENT ON ELR_MESSAGE.ID = ELR_MESSAGE_NBS_PATIENT.ELR_MES
    SAGE_ID
  WHERE ELR_MESSAGE.MSH_10 IS NOT NULL AND
    ELR_MESSAGE_NBS_PATIENT.OBS_LOCAL_ID IS NULL AND
    ELR_MESSAGE.INSERT_DATE >= $DateRange</sql>
</statement>
</config>

```

Destinations

Name	Page
 js Create strMsgControlIDs Property for NBS_ODSE query..... JavaScript (v.1)	332
Error Connector	Page
 no-op to Sink..... No-operation	331

**no-op NBS_ODSE Results**
No-operation
On ECLIR - ELR Polling NBS Patient route

Filter

Notes
Purpose: This No-operation filter receives query results from <i>db NBS_ODSE Query</i> for both results found and empty results.

Destinations

Name	Page
 js Combine Query Responses..... JavaScript (v.1)	326

**no-op REH to Sink**
No-operation
On ECLIR - ELR Polling NBS Patient route

Filter

Notes
Purpose: This No-operation filter sends messages from the <i>ECLIR - ELR Polling NBS Patient Route Error Handler</i> to the <i>ECLIR - ELR Route Error Handler Sink</i>.

Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink..... Sink	13

**no-op to Sink**
No-operation

Filter

On ECLIR - ELR Polling NBS Patient route

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - ELR Sink* after queries are completed or if empty result sets are received.

Destinations

Name	Page
 ECLIR - ELR Sink.....Sink	14



js Create strMsgControlIDs Property for NBS_ODSE query

JavaScript (v.1)

On ECLIR - ELR Polling NBS Patient route

Filter

Notes

Purpose: This JavaScript filter sets the message property strMsgControlIDs to be used in the Database Lookup filters *db NBS_ODSE Query* to query based on the MSH-10 of the messages.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var strMsgControlIDs = '';

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====

//
=====
```

Javascript

```
//_____ SET MESSAGE PROPERTY strMsgControlIDs (start
)_____
//
=====

for (var i=1; i <= next.getRepeatCount('/message/ELR_MESSAGE_NBS_PATIENT_Record'); i++) {
  if (strMsgControlIDs != '') {
    strMsgControlIDs += '|';
  }
  strMsgControlIDs += next.getField('/message/ELR_MESSAGE_NBS_PATIENT_Record['+i+']/MESSAGE_CON
TROL_ID');
}

next.setProperty('strMsgControlIDs', strMsgControlIDs);

//
=====
//_____ SET MESSAGE PROPERTY strMsgControlIDs (end
)_____
//
=====
```

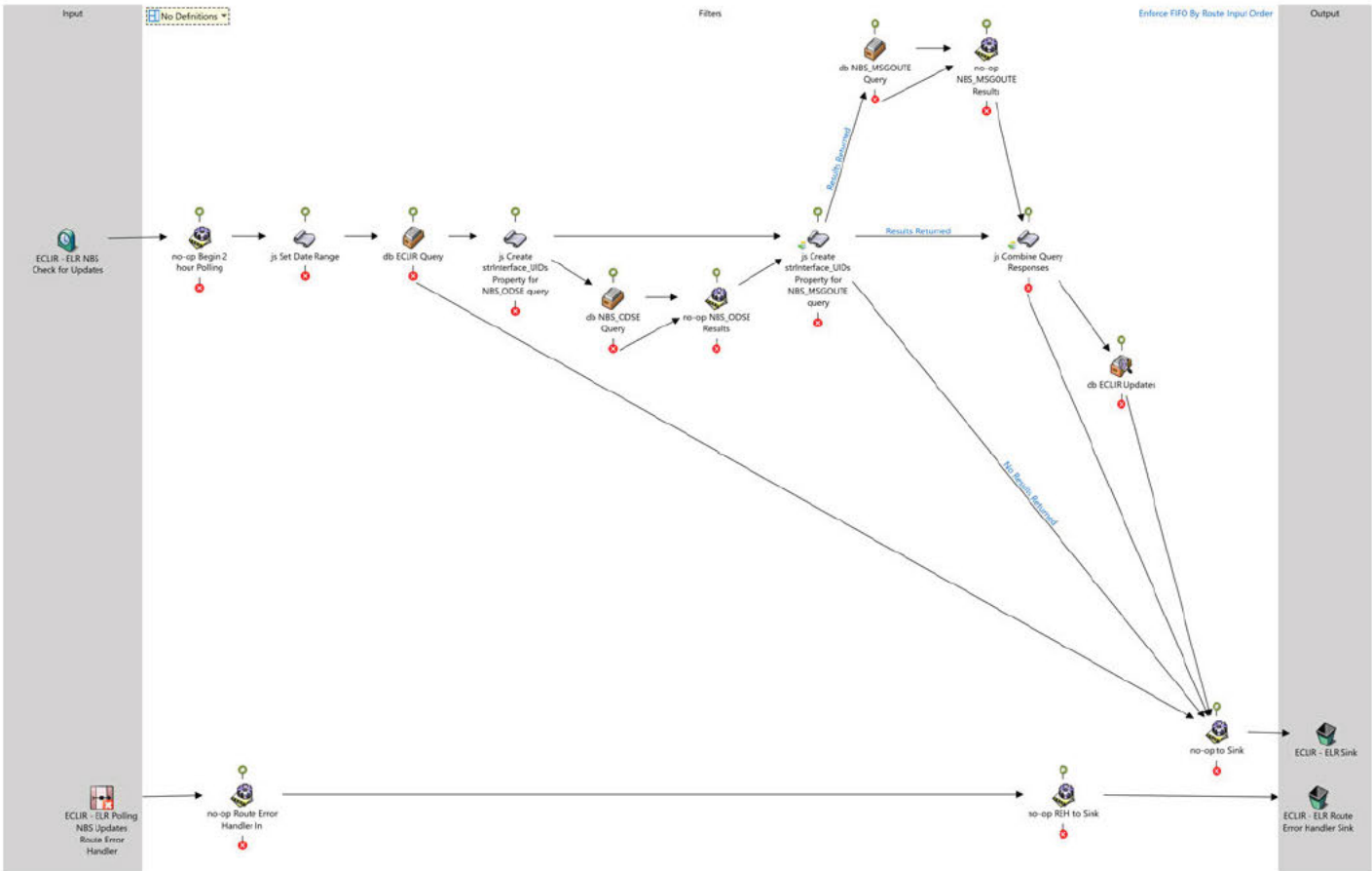
Destinations

Name	Page
 db NBS_ODSE Query..... Database Message Extraction	323
 js Combine Query Responses..... JavaScript (v.1)	326



ECLIR - ELR Polling NBS Updates

Route



Notes

Purpose: This route triggers every 2 hours at 28 minutes past the hour to query NBS tables for key identifiers and stores them to the ECLIR database to allow messages to be linked together throughout the life of the message. This route only runs queries for messages that previously had an NBS status of 'Failure' recorded to see if there have been any updates.

Inputs		Page
ECLIR - ELR NBS Check for Updates.....	Timer	19
Outputs		Page
ECLIR - ELR Sink.....	Sink	14
ECLIR - ELR Route Error Handler Sink.....	Sink	13

Filters

Name		Page
db ECLIR Query.....	Database Message Extraction	346
db ECLIR Updates.....	Database LookUp	344

Name	Page
 db NBS_MSGOUTE Query.....Database Message Extraction	339
 db NBS_ODSE Query..... Database Message Extraction	340
 js Combine Query Responses.....JavaScript (v.1)	336
 js Create strInterface_UIDs Property for NBS_MSGOUTE query.....JavaScript (v.1)	348
 js Create strInterface_UIDs Property for NBS_ODSE query.....JavaScript (v.1)	345
 js Set Date Range.....JavaScript (v.1)	342
 no-op Begin 2 hour Polling.....No-operation	346
 no-op NBS_MSGOUTE Results.....No-operation	338
 no-op NBS_ODSE Results.....No-operation	339
 no-op REH to Sink.....No-operation	350
 no-op Route Error Handler In.....No-operation	343
 no-op to Sink.....No-operation	350

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



js Combine Query Responses

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This JavaScript filter:

Combines the results from the Database Message Extraction filter *db NBS_MSGOUTE Query* and the output from the JavaScript filter *js Create strInterface_UIDs Property for NBS_MSGOUTE query* into an XML file that will be used in the Database Lookup filter *db ECLIR Inserts* to insert any result data received.

A message collector is used to capture the results from:

js Create strInterface_UIDs Property for NBS_MSGOUTE query JavaScript filter

db NBS_MSGOUTE Query Database Message Extraction filter

Several elements are persisted from the *js Create strInterface_UIDs Property for NBS_MSGOUTE query* filter output and added to the *db NBS_MSGOUTE Query* filter results.

If no results are returned from the *db NBS_MSGOUTE Query* filter, the values from the *db NBS_ODSE Query* filter results are transferred over to an output XML in the same structure that would have been returned from the *db NBS_MSGOUTE Query* filter if there had been results.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====

var dbNBS_MSGOUTE_Results = output.append(input[0]);
var dbNBS_ODSE_Results = input[1];

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====

//
=====
//_____ COMBINE QUERY RESPONSES FROM NBS_ODSE AND NBS_MSGOUTE (start
)
//
=====

if (dbNBS_MSGOUTE_Results.text.indexOf('NBS_INTERFACE_Record') != -1) {
  for (var i=1; i <= dbNBS_MSGOUTE_Results.getRepeatCount('/message/NBS_INTERFACE_Record'); i++)
  {
    var NBS_Interface_UID = dbNBS_MSGOUTE_Results.getField('/message/NBS_INTERFACE_Record[' + i +
    ']/NBS_Interface_UID');
  }
}
```

Javascript

```

var ELR_MESSAGE_NBS_ID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/ELR_MESSAGE_NBS_ID");
var EDX_ACTIVITY_LOG_STATUS_from_DB = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/EDX_ACTIVITY_LOG_STATUS_from_DB");
var INTERFACE_IMPORT_STATUS_from_DB = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/INTERFACE_IMPORT_STATUS_from_DB");
var EDX_ACTIVITY_LOG_STATUS = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/EDX_ACTIVITY_LOG_STATUS");
var OBS_LOCAL_ID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/OBS_LOCAL_ID");
var OBS_UID = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/OBS_UID");
var OBS_RECORD_STATUS_CODE = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/OBS_RECORD_STATUS_CODE");
var OBS_DOMAIN_CODE_ST = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/OBS_DOMAIN_CODE_ST");
var Accession_nbr = dbNBS_ODSE_Results.getField("/message/EDX_ACTIVITY_LOG_Record[Interface_UID='"+ NBS_Interface_UID + "']/Accession_nbr");

dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/ELR_MESSAGE_NBS_ID', ELR_MESSAGE_NBS_ID);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS_from_DB', EDX_ACTIVITY_LOG_STATUS_from_DB);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/INTERFACE_IMPORT_STATUS_from_DB', INTERFACE_IMPORT_STATUS_from_DB);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS', EDX_ACTIVITY_LOG_STATUS);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_LOCAL_ID', OBS_LOCAL_ID);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_UID', OBS_UID);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_RECORD_STATUS_CODE', OBS_RECORD_STATUS_CODE);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_DOMAIN_CODE_ST', OBS_DOMAIN_CODE_ST);
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Accession_nbr', Accession_nbr);
}
}
//If no results are returned from the NBS_MSGOUTE query, create a message to only update from NBS_ODSE query results.
else {
dbNBS_MSGOUTE_Results.text = '<message></message>';
for (var i=1; i <= dbNBS_ODSE_Results.getRepeatCount('/message/EDX_ACTIVITY_LOG_Record'); i++)
{
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/ELR_MESSAGE_NBS_ID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/ELR_MESSAGE_NBS_ID'));
var NBS_Interface_UID = dbNBS_MSGOUTE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Interface_UID');
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS_from_DB', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/EDX_ACTIVITY_LOG_STATUS_from_DB'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/INTERFACE_IMPORT_STATUS_from_DB', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/INTERFACE_IMPORT_STATUS_from_DB'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/EDX_ACTIVITY_LOG_STATUS', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/EDX_ACTIVITY_LOG_STATUS'));
});
}

```

Javascript

```

dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/INTERFACE_IMPORT_STATUS', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/INTERFACE_IMPORT_STATUS_from_DB'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_LOCAL_ID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_LOCAL_ID'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_UID', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_UID'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_RECORD_STATUS_CODE', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_RECORD_STATUS_CODE'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/OBS_DOMAIN_CODE_ST', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/OBS_DOMAIN_CODE_ST'));
dbNBS_MSGOUTE_Results.setField('/message/NBS_INTERFACE_Record['+i+']/Accession_nbr', dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Accession_nbr'));
}
}

//
=====
// _____ COMBINE QUERY RESPONSES FROM NBS_ODSE AND NBS_MSGOUTE (end)
// _____
=====

```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR Polling NBS Updates/2b31fc8b-5a0b-4a7c-89dd-9113d142b3f2 ,ECLIR - ELR Polling NBS Updates/c8fca7c1-dfb6-4b42-8663-22682d8ce394]
Match Property	InputTime
Message Set Timeout	2
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
 db ECLIR Updates..... Database LookUp	344
Error Connector	Page
 no-op to Sink..... No-operation	350

**no-op NBS_MSGOUTE Results**

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter receives query results from *db NBS_MSGOUTE Query* for both results found and empty results.

Destinations

Name	Page
js Combine Query Responses..... JavaScript (v.1)	336



no-op NBS_ODSE Results

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter receives query results from *db NBS_ODSE Query* for both results found and empty results.

Destinations

Name	Page
js Create strInterface_UIDs Property for NBS_MSGOUTE query..... JavaScript (v.1)	348



db NBS_MSGOUTE Query

Filter

Database Message Extraction

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This Database Message Extraction filter:

Returns an XML list of records with key identifiers from the NBS_MSGOUTE.nbs_interface table.

If the query returns no records, the message continues from the Error Connector of the filter in order to prevent the message collector in the *js Combine Query Responses* JavaScript filter from timing out.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]

Configuration Properties	
Configuration File	
Message Body Column	
Message Type	XML
Definition	
Query Timeout	
Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0
Configuration File:	
<pre> <statement name="NBS_INTERFACE_Record" ide:type="executeSql" ide:summary="Query NBS_MSGOUTE .NBS_INTERFACE table for key identifiers." ide:allowCompoundStatements="true"> <sql>SELECT ifc.nbs_interface_uid as NBS_Interface_UID, ifc.record_status_cd as INTERFACE_IMPORT_STATUS, ifc.payload as NBS_XML_PAYLOAD FROM [nbs_msgoute].[dbo].[NBS_interface] ifc WHERE ifc.record_status_time >= DATEADD(dd, -CAST(\$pollingDR AS INT), GETDATE()) AND ifc.NBS_Interface_UID IN (SELECT * FROM STRING_SPLIT(\$strInterface_UIDs, ' '))</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op NBS_MSGOUTE Results.....No-operation	338
Error Connector	Page
 no-op NBS_MSGOUTE Results.....No-operation	338



db NBS_ODSE Query

Database Message Extraction

On ECLIR - ELR Polling NBS Updates route

Filter

Notes

Purpose: This Database Message Extraction filter:

Returns an XML list of records with key identifiers from the NBS_ODSE.edx_activity_log based on the returned list from the previous JavaScript filter *js Create strInterface_UIDs Property for NBS_ODSE query*.

If the query returns no records, the message continues from the Error Connector of the filter in order to prevent the message collector in the *js Create strInterface_UIDs Property for NBS_MSGOUTE query* JavaScript filter from timing out.

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:	[REDACTED]	
[REDACTED]		
<pre><statement name="EDX_ACTIVITY_LOG_Record" ide:type="executeSql" ide:summary="Query EDX_ACTIVITY_LOG table in NBS_ODSE to retrieve key identifiers." ide:allowCompoundStatements="true"> <sql>SELECT elog.Message_id AS Message_Control_ID, elog.source_uid AS Interface_UID, elog.record_status_cd AS EDX_ACTIVITY_LOG_STATUS, obs.local_id AS OBS_LOCAL_ID, obs.observation_uid AS OBS_UID, obs.record_status_cd AS OBS_RECORD_STATUS_CODE, obs.obs_domain_cd_st_1 AS OBS_DOMAIN_CODE_ST, elog.Accession_nbr FROM [nbs_odse].[dbo].[EDX_activity_log] elog LEFT OUTER JOIN [nbs_odse].[dbo].[Observation] obs ON elog.target_uid = obs.observation_uid WHERE elog.doc_type='11648804' AND elog.record_status_time >= DATEADD(dd, -CAST(\$pollingDR AS INT), GETDATE()) AND elog.source_uid IN (SELECT * FROM STRING_SPLIT(\$strInterface_UIDs, ' '))</sql> </statement> </config></pre>		

Destinations

Name	Page
 no-op NBS_ODSE Results.....No-operation	339
Error Connector	Page
 no-op NBS_ODSE Results.....No-operation	339



js Set Date Range

JavaScript (v.1)

On ECLIR - ELR Polling NBS Updates route

Filter

Notes

Purpose: This JavaScript filter sets the message property DateRange to be used in the Database Lookup filters *db ECLIR Query* to allow the query to only select records from the last 30 days for querying.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var pollingDR = getVariable('DR_ELR_Updates');
next.text = 'ELR NBS Updates Polling';

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====

//
=====
//_____ FUNCTIONS (start
)
//
=====

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
    num = num.toString();
    while (num.length < size) num = "0" + num;
    return num;
}

//
=====
//_____ FUNCTIONS (end
)
//
=====
```

Javascript

```
//
=====
//_____ SET MESSAGE PROPERTY DateRange (start
)_____
//
=====

var dateRange = new Date();
dateRange.setDate(dateRange.getDate() - pollingDR);
var fieldYear = dateRange.getFullYear();
var fieldMonth = fnPadZeros(dateRange.getMonth() + 1, 2);
var fieldDay = fnPadZeros(dateRange.getDate(), 2);
var fieldHours = fnPadZeros(dateRange.getHours(), 2);
var fieldMinutes = fnPadZeros(dateRange.getMinutes(), 2);
var fieldSeconds = fnPadZeros(dateRange.getSeconds(), 2);

var formatteddateRange = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+fieldMinutes
+':'+fieldSeconds;

next.setProperty('DateRange',formatteddateRange);
next.setProperty('pollingDR',pollingDR);

//
=====
//_____ SET MESSAGE PROPERTY DateRange (end
)_____
//
=====
```

Destinations

Name	Page
 db ECLIR Query..... Database Message Extraction	346



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Polling NBS Updates Route Error Handler* when an unhandled error occurs anywhere in the route. These errors cannot be recorded in the ELR_ERROR table because there will be multiple messages and no way to know which message caused the error.

Destinations

Name	Page
 no-op REH to Sink..... No-operation	350



db ECLIR Updates

Filter

Database LookUp

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This Database Lookup filter updates records in the ELR_MESSAGE_NBS table with key NBS identifiers if any statuses have changed.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File:

```
<statement name="ELR_MESSAGE_NBS Update" ide:type="executeSql" iterate="NBS_INTERFACE_Record"
test="NotEqual(`/message/NBS_INTERFACE_Record/EDX_ACTIVITY_LOG_STATUS_from_DB`, `/message/NBS
_INTERFACE_Record/EDX_ACTIVITY_LOG_STATUS`) or&#xD;&#xA;NotEqual(`/message/NBS_INTERFACE_Record
/INTERFACE_IMPORT_STATUS_from_DB`, `/message/NBS_INTERFACE_Record/INTERFACE_IMPORT_STATUS`)"
ide:summary="Update result data into ELR_MESSAGE_NBS table if any statuses have changed." ide
:allowCompoundStatements="true">
  <sql>UPDATE ELR_MESSAGE_NBS SET
EDX_ACTIVITY_LOG_STATUS = `/message/NBS_INTERFACE_Record/EDX_ACTIVITY_LOG_STATUS`,
```

Configuration File:

```

OBS_LOCAL_ID = `/message/NBS_INTERFACE_Record/OBS_LOCAL_ID`,
OBS_UID = `/message/NBS_INTERFACE_Record/OBS_UID`,
OBS_RECORD_STATUS_CODE = `/message/NBS_INTERFACE_Record/OBS_RECORD_STATUS_CODE`,
OBS_DOMAIN_CODE_ST = `/message/NBS_INTERFACE_Record/OBS_DOMAIN_CODE_ST`,
ACCESSION_NBR = `/message/NBS_INTERFACE_Record/Accession_nbr`,
INTERFACE_IMPORT_STATUS = `/message/NBS_INTERFACE_Record/INTERFACE_IMPORT_STATUS`,
NBS_XML_PAYLOAD = `/message/NBS_INTERFACE_Record/NBS_XML_PAYLOAD`
WHERE ID = `/message/NBS_INTERFACE_Record/ELR_MESSAGE_NBS_ID`</sql>
</statement>
</config>

```

Destinations

Name	Page
 no-op to Sink..... No-operation	350



js Create strInterface_UIDs Property for NBS_ODSE query

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This JavaScript filter sets the message property strInterface_UIDs to be used in the Database Message Extraction filter *db NBS_ODSE Query* to query based on the Interface_UIDs of the messages.

Javascript

```

//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var strInterface_UIDs = '';

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

```

Javascript

```
//
=====
// _____ SET MESSAGE PROPERTY strInterface_UIDs (start
) _____
//
=====

for (var i=1; i <= next.getRepeatCount('/message/ELR_MESSAGE_NBS_Failed_Record'); i++) {
  if (strInterface_UIDs != '') {
    strInterface_UIDs += '|';
  }
  strInterface_UIDs += next.getField('/message/ELR_MESSAGE_NBS_Failed_Record['+i+']/INTERFACE
_UID');
}

next.setProperty('strInterface_UIDs', strInterface_UIDs);

//
=====
// _____ SET MESSAGE PROPERTY strInterface_UIDs (end
) _____
//
=====
```

Destinations

Name	Page
 db NBS_ODSE Query..... Database Message Extraction	340
 js Create strInterface_UIDs Property for NBS_MSGOUTE query..... JavaScript (v.1)	348



no-op Begin 2 hour Polling

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter receives a trigger message every 2 hours from the *ECLIR - ELR NBS Check for Updates* Timer to begin the polling process for ELR NBS updates.

Destinations

Name	Page
 js Set Date Range..... JavaScript (v.1)	342



db ECLIR Query

Filter

Database Message Extraction

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This Database Message Extraction filter returns an XML list of records from the ELR_MESSAGE_NBS table that have key NBS identifiers recorded but did not have a status of 'Success' in either the INTERFACE_IMPORT_STATUS or EDX_ACTIVITY_LOG_STATUS columns. This list will be used to check for any status updates in NBS.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File:

```

<statement name="ELR_MESSAGE_NBS_Failed_Record" ide:type="executeSql" ide:summary="Query to
ELR_MESSAGE_NBS table for all records in the last 30 days previously recorded as failures to
check for an update." ide:allowCompoundStatements="true">
  <sql>SELECT ID AS ELR_MESSAGE_NBS_ID
,EDX_ACTIVITY_LOG_STATUS AS EDX_ACTIVITY_LOG_STATUS_from_DB
,INTERFACE_UID
,INTERFACE_IMPORT_STATUS AS INTERFACE_IMPORT_STATUS_from_DB
FROM ELR_MESSAGE_NBS
WHERE (INTERFACE_IMPORT_STATUS != 'Success' OR EDX_ACTIVITY_LOG_STATUS != 'Success')
AND INSERT_DATE >= $DateRange</sql>
</statement>
</config>

```

Destinations

Name	Page
 js Create strInterface_UIDs Property for NBS_ODSE query.....JavaScript (v.1)	345
Error Connector	Page
 no-op to Sink.....No-operation	350



js Create strInterface_UIDs Property for NBS_MSGOUTE query

Filter

JavaScript (v.1)

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This JavaScript filter:
Sets the message property strInterface_UIDs to be used in the Database Lookup filter *db NBS_MSGOUTE Query* to query based on the Interface_UIDs returned from the *js Create strInterface_UIDs Property for NBS_ODSE query* JavaScript filter.
A message collector is used to capture the results from:
js Create strInterface_UIDs Property for NBS_ODSE query JavaScript filter
db NBS_ODSE Query Database Message Extraction filter
Several elements are persisted from the *js Create strInterface_UIDs Property for NBS_ODSE query* filter output and added to the *db NBS_ODSE Query* filter results.
If no results were returned from the Database Message Extraction filter *db NBS_ODSE Query*, the message is sent to the Sink *ECLIR - ELR Sink*.

Javascript

```
//
=====
//_____ INITIAL VARIABLES (start
)
//
=====

var dbNBS_ODSE_Results = output.append(input[0]);
var dbECLIR_Results = input[1];
var strInterface_UIDs = '';

dbNBS_ODSE_Results.setProperty('NoResults', false);

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====
```

Javascript

```
//
=====
//_____ SET MESSAGE PROPERTY strInterface_UIDs AND PERSIST RESULTS FROM PREVIOUS QUERY (start)
//
=====

if (dbNBS_ODSE_Results.text.indexOf('EDX_ACTIVITY_LOG_Record') != -1) {
  for (var i=1; i <= dbNBS_ODSE_Results.getRepeatCount('/message/EDX_ACTIVITY_LOG_Record'); i++)
  {
    if (strInterface_UIDs != '') {
      strInterface_UIDs += '|';
    }
    var Interface_UID = dbNBS_ODSE_Results.getField('/message/EDX_ACTIVITY_LOG_Record['+i+']/Interface_UID');
    strInterface_UIDs += Interface_UID;
    dbNBS_ODSE_Results.setField('/message/EDX_ACTIVITY_LOG_Record['+i+']/ELR_MESSAGE_NBS_ID', dbECLIR_Results.getField("/message/ELR_MESSAGE_NBS_Failed_Record[INTERFACE_UID='"+ Interface_UID + "']/ELR_MESSAGE_NBS_ID"));
    dbNBS_ODSE_Results.setField('/message/EDX_ACTIVITY_LOG_Record['+i+']/EDX_ACTIVITY_LOG_STATUS_from_DB', dbECLIR_Results.getField("/message/ELR_MESSAGE_NBS_Failed_Record[INTERFACE_UID='"+ Interface_UID + "']/EDX_ACTIVITY_LOG_STATUS_from_DB"));
    dbNBS_ODSE_Results.setField('/message/EDX_ACTIVITY_LOG_Record['+i+']/INTERFACE_IMPORT_STATUS_from_DB', dbECLIR_Results.getField("/message/ELR_MESSAGE_NBS_Failed_Record[INTERFACE_UID='"+ Interface_UID + "']/INTERFACE_IMPORT_STATUS_from_DB"));
  }

  dbNBS_ODSE_Results.setProperty('strInterface_UIDs', strInterface_UIDs);
}
//If no results are returned from the NBS_ODSE query, send to Sink.
else {
  dbNBS_ODSE_Results.setProperty('NoResults', true);
}

//
=====
//_____ SET MESSAGE PROPERTY strInterface_UIDs AND PERSIST RESULTS FROM PREVIOUS QUERY (end)
//
=====
```

Message Collector: Input Collation

Collection Order	[ECLIR - ELR Polling NBS Updates/324849d1-086e-4a39-b9ee-1d4cdbc2c8ba ,ECLIR - ELR Polling NBS Updates/627f0fde-6cc8-4931-98e6-4b2d95712d9d]
Match Property	InputTime
Message Set Timeout	2
Timeout Units	Minutes
Timeout Destination	Process as Usual